STRIDE Threat Model: SAFU Protection Pool (Solana, USDC)

Prepared for: the security auditor
System: SAFU protection pool: one Anchor program (Rust → SBF), its rule library, and its off-chain signers
Repository: `github.com/mrkanchwala/safu_solana` (public), branch `solana-usdc-pool`
Tree modelled: program code at commit `691d1e3` (unchanged at the branch tip); not yet deployed. A frozen tag follows the devnet run
Program source: `programs/safu_pool/src` and `crates/pool-core/src`: 7,475 functional SLOC
Date: 2026-10-23
Framework: STRIDE, the same format as the threat model for the Stellar pool (2026-10-20)
Status: self-assessment prepared as an audit input. This is not an audit.

What are we working on?

Project description

SAFU is a **loss-socialization protocol**: not insurance, and it collects no premium.

A staker deposits USDC into a shared protection pool, nominates a beneficiary, and registers up to three wallets to cover, on Solana, Ethereum or Stellar. If a covered wallet is later drained (by phishing, an approval exploit, or key compromise), SAFU's off-chain fraud scanner scores the incident, and the pool pays out a deterministic entitlement of up to a tier multiple of the stake. The staker's principal is **forfeited** on a paid claim. Backers add capital that counts toward the pool's capacity, earns yield and is never forfeited. There is no premium, no underwriter, no human vote, and no discretionary claims committee.

The program modelled here is the on-chain half, and the Solana build of the pool already modelled for Stellar: the same rules, the same three roles and the same settings. It holds staker and backer principal in a USDC token account owned by a program address, enforces the solvency rule `total_allocated + entitlement ≤ total_staked + total_backed` at admission, verifies the oracle's Ed25519 attestation of a loss event, keeps a first-come line for claims that do not fit today, and streams approved payouts under cooldown, vesting and a daily payout cap. Idle cash is lent on Kamino Lend. Users on Ethereum and Stellar stake without a Solana wallet: they burn USDC through Circle CCTP v2, the mint lands in an identity account the pool derives from their home address, and every later action on that identity needs their own home-chain signature.

Three things differ in shape from the Stellar pool and drive most of what is new here. Solana instructions see only the accounts a transaction lists, so every account is a possible substitution. A failed cross-program call cannot be caught, so every best-effort call to Kamino is checked in advance. And the upgrade authority is a Squads multisig outside the program.

Out of scope for this model: SAFU's fraud-detection scanner is a separate proprietary system and its code is not in this repository. This model treats the scanner's *output*, a signed verdict, as an input crossing a trust boundary, and models what happens when that output is wrong, delayed, or forged. Also out of scope: Solana consensus and the runtime, Anchor 1.1.2 and SPL Token internals, Kamino Lend (klend), Circle's CCTP programs and attestation service, Squads v4, AWS KMS internals, the website, and the Stellar and Ethereum deployments.

Entities

External entities (outside our control)

	ENTITY	NOTES
C8	**Kamino Lend (klend)** USDC reserve	Upgradeable third-party program holding deployed idle capital as kUSDC. Untrusted.
C9	**USDC mint + SPL Token program**	Circle's USDC, classic SPL Token. Fixed at `initialize`; Token-2022 refused. Trusted.
C10	**Circle CCTP v2** (MessageTransmitter, TokenMessengerMinter, attestation service)	Mints USDC on Solana for a burn on Ethereum or Stellar; checks its own attestation and nonce. Trusted for the mint, nothing else.
C11	**Staker / backer / beneficiary**	End user, on Solana directly or on Ethereum / Stellar through C3. Untrusted input source.
C12	**Solana runtime, RPC, Ed25519 and secp256k1 programs**	Consensus, signatures, the instructions sysvar, native signature checks. Trusted platform.
C13	**Squads v4 multisig**	Holds the program's upgrade authority: the three roles, 2 of 3, 7-day time lock. Trusted platform; its members are C6.

Processes (under our control)

	PROCESS	NOTES
C1	**`safu_pool` program**	The system under review: pool, claims, line, streams, settings, governance, covered-wallet registry, Kamino leg, CCTP. Upgradable only through C13.
C2	**`pool-core` rule library**	Every rule as a pure function, compiled into C1 and read by clients through the IDL.

	PROCESS	NOTES
C3	**CCTP inbox and identity accounts** (inside C1)	An inbox per mode receives the mint; an identity PDA per home owner stakes or backs. Only the owner's Ethereum `personal_sign` or Stellar SEP-53 signature can move its money, and only into the pool or home.
C4	**Upkeep inside every user call** (inside C1)	Releases the head of the line and checks two watch places, using accounts the transaction must carry. No keeper.
C5	**Oracle backend, registry writer and CCTP relayer**	Off-chain scanner and signers. The oracle's Ed25519 key is held in **AWS KMS** and never leaves it. The relayer only pays fees and rent: it cannot sign for an owner.
C6	**Governance roles: admin, co-signer, guardian**	Two of three for every power-granting change, and the Squads members for upgrades. Co-signer key in KMS on the server; guardian key in KMS, used only from AWS CloudShell. Admin alone can pause and suspend within limits.
C7	**Frontend**	The site. Client-side; treated as untrusted.

Data storage (accounts)

ACCOUNT	SEEDS	CONTENTS
`Pool`	`[pool]`	Roles, oracle, registry writer, treasury, USDC mint, cluster, yield venue, deploy share, totals, yield indexes and set-asides, protocol yield, rounding reserve, 28 settings and pending changes, pending governance changes, pause record, line head and tail, watch cursor, day counters, version
Vault	`[vault, pool]`	The pool's USDC token account; authority = the pool PDA

ACCOUNT	SEEDS	CONTENTS
kUSDC account	`[leg, pool, collateral mint]`	The pool's Kamino collateral; authority = the pool PDA
`StakeRecord` / `BackerRecord`	`[stake, pool, staker]` / `[backer, pool, backer]`	Amount, beneficiary, snapshots of the rules at stake time, suspension, forfeiture, yield owed, points; backer pending deposits (up to 8) and open withdrawal
`Claim`	`[claim, pool, staker, tx_hash]`	The claim id is the address; status, entitlement, streamed, clocks, blocked-time mark, line place
`QueueEntry` / `WatchEntry`	`[queue, pool, seq]` / `[watch, pool, seq]`	The claim line and the watch list
`Override` / `RevokedApproval`	`[override, claim]` / `[revoked, pool, hash]`	The two-key override; permanent revocation of a signed approval
`CoveredWallet` / `StakerWallets`	`[covered, pool, wallet_hash]` / `[staker_wallets, pool, staker]`	Wallet hash → staker, for good; up to 3 per staker
`CctpAccount` + identity	per mode and home owner	Home domain, owner, cluster, nonce, `to_deploy`; the identity PDA owns the held USDC account
AWS KMS	off-chain	Oracle Ed25519 key; co-signer and guardian keys; relayer fee-payer key

Every account carries a version and spare bytes. An instruction refuses with `NotMigrated` until `migrate` has brought the pool up to the program's version.

Assets under protection

ASSET	WHY IT MATTERS
Staker and backer principal (USDC), liquid or lent on Kamino	direct financial loss
Solvency at admission `total_allocated + entitlement ≤ capacity`	the protocol's core promise; a breach means not all payouts can be honoured
Entitlement correctness	an inflated entitlement is a socialised loss borne by everyone else in the pool

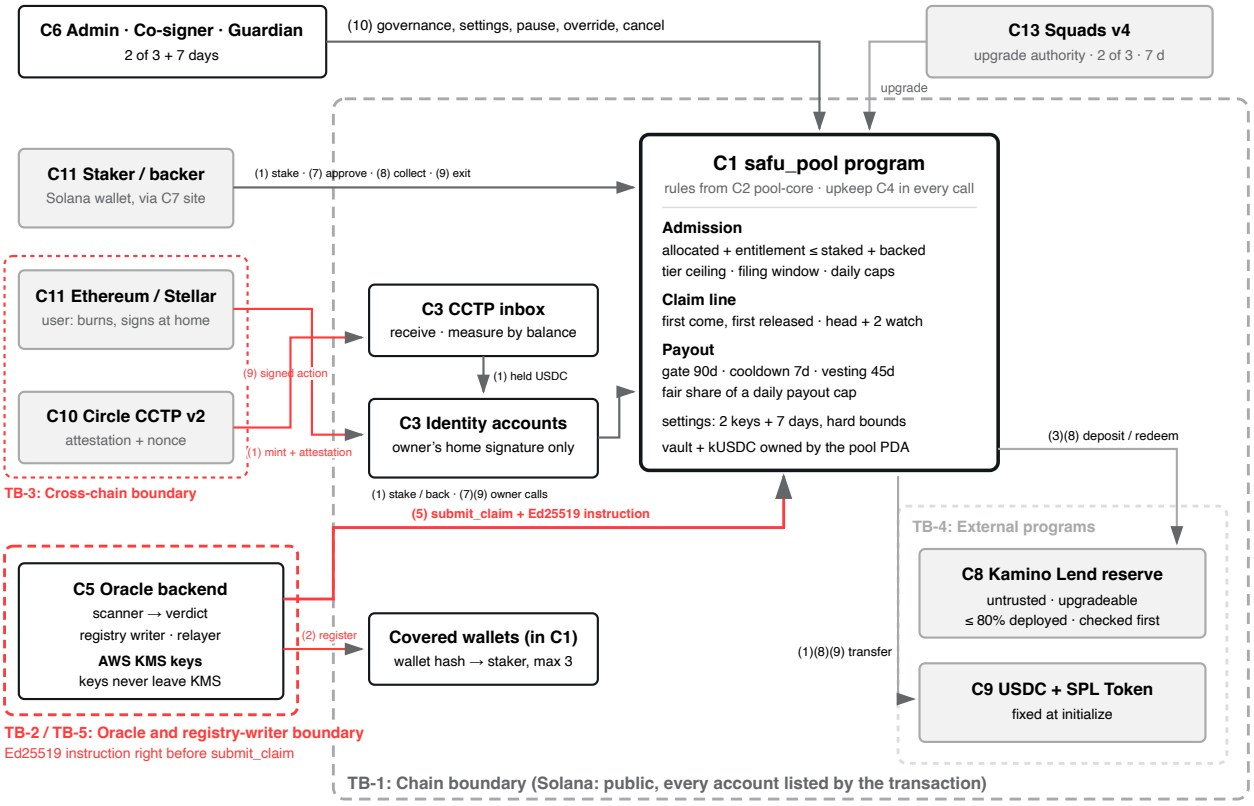
ASSET	WHY IT MATTERS
Exit availability (`withdraw`, `emergency_exit`, backer drip)	funds locked with no exit is materially a loss even without theft
Claim line order	a line that can be jumped turns timing into a payout advantage
Covered-wallet bindings	a wallet covered twice can be claimed twice
CCTP identities and the USDC they hold	an owner's money must move only on the owner's signature
Oracle Ed25519 key, role keys, the upgrade authority	claim admission; settings, venue, treasury, pause, suspension, override, code

Happy-path flow

1. **Stake.** A Solana user calls `stake(amount, beneficiary)`. An Ethereum or Stellar user burns USDC through CCTP naming the pool's inbox; anyone (SAFU's relayer) calls `cctp_receive`, which moves the mint into the owner's held account, then `cctp_deploy`, which stakes or backs it as the owner's identity. A deposit above the maximum stake becomes backing.
2. **Cover.** The owner proves control of each wallet off-chain; the registry writer calls `register_wallet` with the wallet's hash.
3. **Idle capital.** Every deposit pushes idle cash above the buffer into the Kamino reserve (C8), up to the deploy share (80% at most).
4. **Loss event.** A covered wallet is drained. SAFU's scanner checks coverage and scores the transaction, producing a verdict: entitlement in USDC, tier, hack time.
5. **Attest and submit.** The oracle backend signs the approval message with the KMS-held Ed25519 key. The transaction carries the Ed25519 precompile instruction immediately before `submit_claim`, and the oracle also signs the transaction.
6. **Admit or queue.** The program checks the signer, the precompile instruction and the message; then the tier ceiling, the filing window, solvency, the daily admission cap and the oracle's daily limit. It admits the claim, or puts it at the back of the line.
7. **Staker approves.** After the 90-day time gate the staker calls `approve_claim` (upkeep can also move it forward). The stake is forfeited, points are burned, and a 7-day cooldown and then linear vesting begin.
8. **Collect.** The beneficiary collects with `claim_stream`, within the claim's fair share of each day's payout cap. The program pulls cash back from Kamino first if needed.
9. **Or exit.** The staker calls `withdraw` (to the committed beneficiary), or `emergency_exit` while paused. Backers withdraw through a daily drip. Yield can be collected at any time. CCTP owners act with `cctp_act` and bring money home with `cctp_send_home`.

10. **Upkeep and exceptions.** Every user instruction ends with upkeep over the accounts it carries. Exceptions: admin `cancel_claim`; admin + co-signer `approve_override`; governance changes and settings through their delays; upgrades through Squads; permissionless expiries and line release.

Data flow diagram



Numbers in parentheses refer to the happy-path flow steps above.
Solid dark outline = process under our control. Grey = external (entity or program).
Red = a signed input crossing a trust boundary (oracle verdict, registry write, CCTP attestation, owner signature).

Trust boundaries

- **TB-1: chain boundary.** Everything crossing it is public and ordered by consensus. On Solana the transaction, not the program, chooses which accounts an instruction sees, so every account is checked for owner, type, seeds and its relation to the pool before it is trusted.
- **TB-2: oracle attestation boundary.** The one place an off-chain judgement is admitted on-chain. The verdict arrives as an Ed25519 precompile instruction in the same transaction, which the runtime verifies; the program then proves it is the right instruction, signer and message.
- **TB-3: cross-chain boundary.** A CCTP message with Circle's attestation, and a home-chain owner signature, cross here. Circle's transmitter checks the attestation and refuses a used nonce; the program

checks recipient, caller, source domain and decimals, and checks every owner signature itself.

- **TB-4: external program boundary.** `C1 → C8`, `C1 → C9`, `C1 → C10`. Program ids are pinned; klend's accounts are checked against the reserve before any call, and its returns are measured by balance change.
- **TB-5: registry-writer boundary.** One key writes covered-wallet bindings after an off-chain ownership proof. The scanner, not the pool, checks coverage before the oracle signs.

What can go wrong?

Threat table

THREAT	ISSUES
Spoofing	**Spoof.1**: (Step 5) An attacker submits a claim while impersonating the oracle, admitting a fabricated loss event. **Spoof.2**: (Step 5) An attacker passes a forged or misplaced Ed25519 instruction, or a fake instructions sysvar, so the program reads a signature the runtime never checked. **Spoof.3**: (Steps 7–9) An attacker impersonates a staker to approve a claim, collect a payout or withdraw principal to an address of their choosing. **Spoof.4**: (Steps 1, 9) An attacker forges an Ethereum or Stellar owner's signature, replays one, or re-splits a signed action into other data and accounts, to act through that owner's identity. **Spoof.5**: (Step 1) An attacker forges or redirects a CCTP mint, crediting a fake deposit or someone else's identity. **Spoof.6**: (all) An attacker passes a look-alike account (wrong owner, wrong type, wrong seeds, a fake klend reserve, another mint) in place of the one the instruction expects. **Spoof.7**: (Step 10) An attacker impersonates a governance role, or a role key is installed at a mistyped address. **Spoof.8**: (deploy) An attacker calls `initialize` first on a freshly deployed program.
Tampering	**Tamper.1**: (Step 5) The signed verdict is altered so that entitlement, tier, staker or transaction differ from what the scanner produced. **Tamper.2**: (Step 5) A signature is replayed on another cluster, against another program, or twice on the same pool.

THREAT	ISSUES
	Tamper.3: (Step 10) A settings change re-prices stakes and claims already in the pool. **Tamper.4**: (Steps 3, 8) Kamino reports a false exchange rate or liquidity, so the pool over-books on deposit or under-receives on redeem. **Tamper.5**: (Steps 8–9) Arithmetic is manipulated through ordering, truncation or rounding in tier ceilings, yield indexes, daily shares or klend conversions. **Tamper.6**: (Step 2) A covered wallet is rebound to another staker, or covered again after it was claimed on. **Tamper.7**: (Step 6) The claim line is reordered: a later claim is released before an earlier one. **Tamper.8**: (all) An account is re-initialised, or a closed account revived, to reset its state.
Repudiation	**Repudiate.1**: (Steps 5–9) A payout, forfeiture, or claim-state transition cannot be attributed after the fact. **Repudiate.2**: (Step 10) A role changes a setting, the oracle, the venue, the treasury, the registry writer or the code, or pauses the pool, and no monitor can detect it. **Repudiate.3**: (Steps 4–5) The oracle denies having attested to a verdict.
Information Disclosure	**Info.1**: (Step 1) The beneficiary address is visible on-chain. **Info.2**: (Step 2) Covered wallets are readable on-chain, telling an attacker which wallets are protected. **Info.3**: (Step 4) Scanner thresholds or scoring logic are inferable from on-chain data. **Info.4**: (Step 6) Stake amounts, claim history and payouts are publicly readable.
Denial of Service	**DoS.1**: (Steps 5, 9) A compromised oracle files claims against many stakers at once; a staker with an open claim cannot exit. **DoS.2**: (Step 6) Claim flooding, or a mass incident,

THREAT	ISSUES
	crowds genuine claims out. **DoS.3**: (Step 8) A coordinated payout run drains the pool faster than it can absorb. **DoS.4**: (Steps 5–8) A pause, or days Kamino cannot pay, run out a claim's filing, approval or collection window. **DoS.5**: (Steps 8–9) A payment cannot be funded because cash is lent out, or a failed Kamino call aborts the user's whole transaction. **DoS.6**: (all) Upkeep accounts, compute or transaction size are used to make user calls fail. **DoS.7**: (Step 1) A CCTP deposit is lost when the pool refuses it. **DoS.8**: (Step 10) An endless pause holds every claim.
Elevation of Privilege	**Elevation.1**: (all) The program is upgraded to a malicious build that drains principal. **Elevation.2**: (Step 10) One role takes over the others. **Elevation.3**: (Step 10) The admin turns configuration into theft of principal. **Elevation.4**: (Step 10) The 2-of-2 override pays a colluding claimant. **Elevation.5**: (Steps 7–9) A false claimant whose claim is cancelled keeps what was paid and leaves at once with the full stake. **Elevation.6**: (Steps 3, 8) The pool PDA's signing authority is used in a call to a program other than the one intended. **Elevation.7**: (Step 1) The beneficiary is set to a privileged party, the pool or its vault. **Elevation.8**: (deploy) A fast-clock test build runs as a real pool.

What are we going to do about it?

Every mitigation below is **implemented in the tree being submitted for audit** unless explicitly labelled **ACCEPTED RISK** or **OPERATIONAL**.

THREAT	ISSUES AND REMEDIATIONS
Spoofing	**Spoof.1**: impersonating the oracle. **Spoof.1.R.1**: `submit_claim` requires the stored oracle as a transaction signer (`has_one = oracle`). **Spoof.1.R.2**: the Ed25519 signature must be by the oracle's key over the full approval message. The transaction signer binds *who submitted*, the signature binds *who attested*. **Spoof.1.R.3**: the oracle's daily limit (one tenth of stakers, at least one) caps how many claims it can admit in a day. **Spoof.2**: forged or misplaced Ed25519 instruction. **Spoof.2.R.1**: the instructions sysvar is checked by address. **Spoof.2.R.2**: the precompile instruction must sit **immediately before** `submit_claim` , at the top level, carry exactly one signature, and keep every offset inside its own data, so no part of the checked signature can point elsewhere. **Spoof.2.R.3**: the message is rebuilt on-chain from the instruction arguments and compared byte for byte; a backend test vector pins the layout. **Spoof.3**: impersonating a staker. **Spoof.3.R.1**: `approve_claim` , `withdraw` , `set_beneficiary` and yield claims require the staker's signature; `claim_stream` , `withdraw` and `claim_yield` pay only the beneficiary stored on the stake. **Spoof.3.R.2**: `set_beneficiary` is refused while a claim is open or queued, so a key stolen in the drain cannot redirect the payout. **Spoof.4**: forging a home-chain owner. **Spoof.4.R.1**: Ethereum: secp256k1 recovery over the EIP-191 digest, recovery id checked, address compared with the stored owner. Stellar: SEP-53 signed message checked by the Ed25519 instruction right before the call. **Spoof.4.R.2**: the signed digest covers the program id, the cluster, the owner's CCTP account, a nonce, an expiry of at most 1 hour, and a length-prefixed, domain-separated

THREAT	ISSUES AND REMEDIATIONS
	hash of the call data and of every account it uses with their count. A signature cannot be re-split into other data and accounts (found in internal review, fixed, regression test kept). **Spoof.4.R.3**: each nonce works once. Only 11 pool calls are allowed, and `set_beneficiary` is not one; a stake's beneficiary must be the identity itself. **Spoof.5**: forged or redirected CCTP mint. **Spoof.5.R.1**: Circle's transmitter verifies the attestation and refuses a used nonce. **Spoof.5.R.2**: the burn must name the pool's inbox as recipient and its CCTP authority as the only allowed caller; only Ethereum and Stellar are accepted as source domains; a local mint with fewer decimals than Circle's canonical 6 is refused. **Spoof.5.R.3**: the identity is derived from the message's sender, and the amount is the measured balance change, never a field in the message. **Spoof.6**: look-alike accounts. **Spoof.6.R.1**: typed Anchor accounts check owner and discriminator; PDAs are checked by seeds and stored bumps; token accounts are checked for mint (`pool.usdc_mint`) and authority. **Spoof.6.R.2**: klend accounts: program id pinned; the reserve must be `Pool.yield_venue`; owner, discriminator and length are checked before any byte is read; market, market authority, supply vault, collateral mint, the kUSDC account and the one price account must match what the reserve records. **Spoof.6.R.3**: the USDC mint must be owned by classic SPL Token with 6 decimals (Token-2022 refused at `initialize`). **Spoof.7**: impersonating a role. **Spoof.7.R.1**: every propose, approve, cancel and settings step requires the acting role's signature; the role a change replaces never counts. **Spoof.7.R.2**: a new role, oracle or registry-writer key signs its own installation; the three roles and the oracle must be four different keys. **Spoof.8**: front-running `initialize`. **Spoof.8.R.1**: `initialize` requires the program's upgrade authority to sign. The deployer runs it, then hands the authority to Squads.

THREAT	ISSUES AND REMEDIATIONS
Tampering	**Tamper.1**: verdict altered. **Tamper.1.R.1**: every decision field is inside the signed message: domain, program id, cluster, staker, drain transaction hash, entitlement, tier, hack time, deadline. Every field is fixed-size, so the encoding is unambiguous. **Tamper.2**: replay. **Tamper.2.R.1**: the program id and the pool's cluster tag are signed, so a signature is valid for one program on one cluster. **Tamper.2.R.2**: the claim account `[claim, pool, staker, tx_hash]` is the nonce: a used, cancelled or expired claim can never be filed again. The deadline is at most 24 hours ahead. **Tamper.2.R.3**: the admin can revoke a signed approval before use; the revocation is a permanent program-owned account, and lamports sent to its address do not block a valid approval. **Tamper.3**: settings re-pricing people already in. **Tamper.3.R.1**: tier ratios, time gate and filing window are copied onto a stake; approval and inactivity windows, cooldown and vesting onto a claim. A cut to a ceiling applies to new stakes only. **Tamper.3.R.2**: 28 settings, each with hard bounds and order rules, checked at proposal and again at execution, after a 7-day public timelock. **Tamper.4**: Kamino reports false values. **Tamper.4.R.1**: kUSDC received and USDC returned are measured by balance change; book value is never marked to market. **Tamper.4.R.2**: growth is booked only when it comes back, capped at 10 bp a day of book value; keeper moves accept at most 5% below book; only the admin redeems lower, with an explicit minimum. **Tamper.4.R.3**: a reserve priced by two sources, or by Switchboard, is not a venue; a venue can be changed only through governance and never while anything is deployed. **Tamper.5**: arithmetic manipulation. **Tamper.5.R.1**: every rule is a pure function in `pool-core` with unit and property tests; multiply before divide; payouts round down, collateral needed rounds up. **Tamper.5.R.2**: the release build aborts on overflow; narrowing casts are on bounded values only.

THREAT	ISSUES AND REMEDIATIONS
	Tamper.5.R.3: a rounding reserve anyone can fund and nothing can take out covers a payment short by at most 0.001 USDC. **Tamper.6**: covered wallet rebound or covered twice. **Tamper.6.R.1**: a `CoveredWallet` account per wallet hash binds it to its staker for good, at most 3 per staker, written only by the registry writer. **Tamper.6.R.2**: a forfeited stake record refuses every new stake, so a drained wallet cannot be covered by the same address twice. **Tamper.7**: line reordering. **Tamper.7.R.1**: queued claims are `QueueEntry` accounts released only at the head, and a new claim must name the back of the line and joins it while anyone waits. Release is permissionless and also runs in upkeep. **Tamper.8**: re-initialisation and revival. **Tamper.8.R.1**: every `init_if_needed` has a guard: a claim must be `Unused`, a backer or registry record must be empty; a re-stake on an empty record keeps its suspension. **Tamper.8.R.2**: claim, queue and override accounts close through Anchor `close`.
Repudiation	**Repudiate.1**: payouts unattributable. **Repudiate.1.R.1**: 56 event types (`events.rs`) cover every money movement and claim transition (submitted, queued, released, approved, streamed, cancelled, expired, overrides), stakes, withdrawals, exits, backer deposits and drips, yield, Kamino deposits, redeems, shortfalls and harvests, and CCTP receive, deploy, act and send home. **Repudiate.1.R.2**: `view_staker` and `view_backer` return the live position, including growth not yet harvested. **Repudiate.2**: administrative action undetectable. **Repudiate.2.R.1**: every governance step, every settings step, pause, unpause, suspension, renewal and treasury payout emits an event. Squads proposals and votes are public accounts. **Repudiate.3**: oracle denies attestation. **Repudiate.3.R.1**: the Ed25519 signature over the full message is in the transaction, verifiable by anyone against

THREAT	ISSUES AND REMEDIATIONS
	the oracle key stored on the pool. **Repudiate.3.R.2**: signing happens inside AWS KMS, which logs every signing call outside the reach of the application host.
Information Disclosure	**Info.1**: beneficiary visible. **ACCEPTED RISK.** **Info.1.R.1**: on Solana the beneficiary is stored as a plain address, since it is public at payout anyway. It cannot be changed while a claim is open (Spoof.3.R.2). **Info.2**: covered wallets readable. **Info.2.R.1**: the program stores only a hash of each covered wallet, not the address. **Info.3**: scanner logic inferable. **Info.3.R.1**: the program receives only a finished verdict; no scoring logic is on-chain or in this repository. **Info.4**: pool state is public. **ACCEPTED RISK.** **Info.4.R.1**: inherent to a transparent ledger and to the pool's own auditability promise.
Denial of Service	**DoS.1**: oracle griefing. **ACCEPTED RISK: bounded, remedy exists and is penalty-free; residual is operational.** **DoS.1.R.1**: bounded by the oracle's daily limit, one tenth of stakers per day. **DoS.1.R.2**: nothing is forfeited until the staker approves their own claim. **DoS.1.R.3**: admin `cancel_claim` clears a claim the staker never approved, with no penalty. **DoS.1.R.4**: **OPERATIONAL:** clearing depends on the admin acting. **DoS.2**: flooding or a mass incident. **DoS.2.R.1**: a claim that does not fit today joins the line instead of failing, and a queued claim never expires on time. **DoS.2.R.2**: every claim needs an oracle signature and a stake past the 90-day gate. **DoS.3**: payout run. **DoS.3.R.1**: admission checks `total_allocated + entitlement ≤ capacity` and a daily admission cap tiered by utilisation. **DoS.3.R.2**: payouts are spread by cooldown, vesting and a daily payout cap; each claim gets a pro-rata share of the day by what it is still owed, from the day's snapshot, so

THREAT	ISSUES AND REMEDIATIONS
	the order of calls doesn't matter. **DoS.4**: windows running out. **DoS.4.R.1**: the approval and collection windows run on a claim clock that does not count paused time or whole days on which a payout found the pool short of cash; every expiry sweep refuses while paused. **DoS.4.R.2**: the hack-to-filing window does not count paused time, and the filing window can never be set longer than the gap between pauses. **DoS.5**: payment unfunded, or a Kamino call aborts the user's call. **DoS.5.R.1**: every payment first pulls the shortfall plus a refill of the buffer from Kamino, in the same instruction, also while paused. **DoS.5.R.2**: Solana cannot catch a failed cross-program call, so every best-effort Kamino step checks in advance everything klend would check (reserve status, the market's emergency flag, deposit limit, cash after queued withdrawal tickets) and skips if any fails. A payout Kamino cannot fund pays nothing and counts a cash-short day; it never aborts. **DoS.5.R.3**: harvest and push are skipped when less than their compute budget is left, so a user call never fails for lack of compute. **DoS.6**: upkeep, compute and size. **DoS.6.R.1**: every loop has a fixed cap: the line head and 2 watch places per call, 8 pending backer deposits, 3 wallets, 20 drain transactions per claim. **DoS.6.R.2**: missing or stale upkeep accounts refuse the call (`UpkeepAccountsMissing`); the site re-reads and resends once. Upkeep is checked before it writes and is skipped while paused. **DoS.6.R.3**: the busiest user call measures about 260k compute units against the site's 400k limit, and the largest transaction with full upkeep fits one 1,232-byte packet (tests). **DoS.7**: CCTP deposit refused. **DoS.7.R.1**: `cctp_receive` never refuses for pool reasons, so a mint can always complete. If `cctp_deploy` is refused, the USDC stays in the owner's held account and can be deployed later or sent home. **DoS.8**: endless pause. **DoS.8.R.1**: a pause lasts at most 30 days and ends on its

THREAT	ISSUES AND REMEDIATIONS
	own; the next pause needs a gap after the last one ended. **DoS.8.R.2**: `emergency_exit`, backer withdrawals and yield claims work while paused.
Elevation of Privilege	**Elevation.1**: malicious upgrade. **ACCEPTED RISK, stated plainly: two of three roles control the pool after 7 days.** **Elevation.1.R.1**: the upgrade authority is a Squads v4 multisig of the three roles, threshold 2, with a 7-day time lock between approval and execution. **Elevation.1.R.2**: stakers can see a pending upgrade for those 7 days and leave. Every account is versioned and every instruction refuses until `migrate` runs, so new code never misreads old data. **Elevation.2**: one role takes over. **Elevation.2.R.1**: roles must be distinct. One remaining role can replace another only after 90 days, and any other working role, including the one being replaced, can cancel it. **Elevation.3**: admin turns configuration into theft. **Elevation.3.R.1**: no instruction sends staker or backer principal anywhere but its owner's committed address; CCTP identities send only into the pool or home to their owner. **Elevation.3.R.2**: the treasury takes only the protocol's own yield balance, capped at the surplus, and never while paused. **Elevation.3.R.3**: a pause or suspension delays, never takes: a suspension by the admin alone lapses after 30 days, and renewing it needs a second role. **Elevation.4**: collusive override. **ACCEPTED RISK, bounded.** **Elevation.4.R.1**: the override needs admin and co-signer, still needs the staker's own approval, and stays inside the tier ceiling, capacity, daily caps, cooldown and vesting. On a forfeited stake only the claim that forfeited it can be re-executed. **Elevation.5**: cancelled false claim keeps its gains. **Elevation.5.R.1**: cancelling an approved claim restores only the stake minus what was already paid; a claim paid up to the stake leaves it forfeited. **Elevation.5.R.2**: the rest is locked for 365 days, on both `withdraw` and `emergency_exit`.

THREAT	ISSUES AND REMEDIATIONS
	Elevation.6: the pool PDA signs for the wrong program. **Elevation.6.R.1**: the pool signs only calls to pinned program ids (SPL Token, klend, Circle CCTP), and the owner path calls only this program, with an 11-call allow-list. **Elevation.7**: privileged or circular beneficiary. **Elevation.7.R.1**: the beneficiary cannot be the oracle, admin, co-signer, the pool or its vault. **Elevation.8**: test build as a real pool. **Elevation.8.R.1**: the fast-clock `demo` build refuses to initialize a mainnet pool.

Summary of open items

ID	SEVERITY	ITEM	DISPOSITION
Elevation.1	n/a	Two of three roles control the pool after 7 days, including upgrades	**Accepted risk**, disclosed (`docs/AUDITOR_DISCLOSURES.md` §2, §3)
Elevation.4	n/a	Collusive override: bounded, observable	**Accepted risk**
DoS.1	LOW-MEDIUM	Oracle griefing (one tenth of stakers per day); cleared by admin `cancel_claim`, penalty-free	**Accepted + disclosed**: operational dependency
Info.1	n/a	The beneficiary is stored as a plain address	**Accepted**: public at payout anyway
Rounding dust	INFO	A Kamino redeem in a pool with backers and no stakes can leave a few base units unbooked	**Disclosed** (§6); backers are never marked down by design
Spoof.4	INFO	No low-s check on Ethereum owner signatures; the nonce stops replay	**Disclosed** (§11)

ID	SEVERITY	ITEM	DISPOSITION
Duplicate mutable accounts	n/a	Not yet proven instruction by instruction that two mutable slots cannot be passed the same account	**For the audit** (§14)
TB-5	n/a	Coverage is checked off-chain before the oracle signs; the pool trusts the signed verdict	**Accepted, by design**: one trust root (the oracle)
Liquidity	n/a	After large claims the last withdrawer can wait for new money	**Accepted, by design** (§1)

Assurance evidence

CONTROL	RESULT	SCOPE NOTE
Program and rule-library tests	**287 passing on each build** (program 200, `pool-core` 87), normal and fast-clock	LiteSVM with klend's mainnet program and Circle's CCTP programs loaded; 2026-10-23
Stateful fuzz	**593,250 steps across both builds, 0 failures** (normal 295,750, fast-clock 297,500)	Build server, capped container, both builds
Mutation testing	Smoke run **26 caught, 0 missed**; full run over the program and `pool-core` in progress	GitHub Actions, `.github/workflows/mutants.yml`; results in `docs/TESTING.md`
`cargo audit`	**0 known vulnerabilities in the program's dependency graph**; 3 advisories reach test-only `litesvm`	2026-10-23
`cargo clippy`, `cargo deny`	0 warnings on both builds; licence and advisory policy in CI	2026-10-23
Internal reviews	Code review, security review, infrastructure review, Solana checklist audit: **0 critical / high**; 7 medium and every low with a fix fixed with tests	2026-10-23

Did we do a good job?

Has the data flow diagram been referenced since it was created?

Yes. Drawing the home-chain owner as a red arrow into the identity accounts, next to the CCTP mint, is what separated the two halves of TB-3: Circle checks the mint, but nobody but the program checks the owner's signature. That question, what exactly the owner signed, led straight to the re-split issue in Spoof.4.R.2.

Did the STRIDE model uncover any new design issues or concerns that had not been previously addressed?

Yes. Walking Spoofing against the CCTP path showed that an owner's signature covered the call data but not which accounts the call used, so a relayer could move the accounts into the data and keep the same signature (internal review, confirmed with a test). The fix hashes the data and every account with their lengths and count under a new domain. Walking Elevation of Privilege against the test build showed that nothing on-chain stopped a fast-clock build from being initialized as a real pool (Elevation.8.R.1). Both are fixed with tests.

Did the treatments adequately address the issues identified?

Mostly, and the exceptions are named. Every Spoofing, Tampering and Repudiation issue is treated in code. What remains is accepted rather than treated: governance by two of three roles including upgrades (Elevation.1), the bounded collusive override (Elevation.4), the operational dependency in DoS.1, the plain beneficiary address (Info.1), and the trust placed in the oracle's coverage check (TB-5). The rounding dust and the missing low-s check are disclosed as harmless. Duplicate mutable accounts are listed for the audit rather than claimed.

Have additional issues been found after the threat model?

The stateful fuzz found one after this model was first drafted: the rounding dust in a pool with no stakes. It is disclosed rather than changed in code (§6), since the amount is a few millionths of a dollar and a new money path would add more risk than it removes. The model will be re-run against the frozen tag and again against the post-audit tree before mainnet.

Any additional thoughts on the threat modelling process?

Writing the Solana model against the Stellar one helped most where the two differ. On Stellar the host decides which contract storage a call can read; on Solana the transaction decides which accounts an instruction sees. Most of the new mitigations here (Spoof.2, Spoof.6, DoS.6) exist because of that one difference.

Prepared by SAFU as a self-assessment for the security audit. Open findings are disclosed here rather than remediated silently; accepted risks are also published in the repository's `docs/AUDITOR_DISCLOSURES.md`*.*